



TOPIC : Static in Java

Static Modifier in Java

Static Modifier in Java (मराठीत)

1. ओळख (Introduction)

Java मध्ये **static** हा एक **non-access modifier** आहे जो **variables, methods, blocks** आणि **nested classes** वर लागू होतो.

static चा अर्थ असा की तो **member object-specific** नसून **class-specific** असतो. याचा अर्थ, **static member** ची **JVM** मध्ये एकच **copy** तयार होते आणि ती सर्व **objects** मध्ये **share** होते.

2. Static Variables (Class Variables)

static variable ला **class variable** देखील म्हणतात कारण ते एखाद्या **object** पेक्षा **class** शी संबंधित असते.

त्याची **value** सर्व **objects** साठी समान (**common**) राहते.

उदाहरण:

```
class Student {  
    static String college = "SP College";  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(name + " - " + college);  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student("Rahul");  
        Student s2 = new Student("Priya");  
  
        s1.display();  
        s2.display();  
  
        Student.college = "New SP College"; // static change  
    }  
}
```

```
s1.display();
s2.display();
}
}
```

Output:

Rahul - SP College
Priya - SP College
Rahul - New SP College
Priya - New SP College

स्पष्टीकरण:

college हे **static variable** असल्यामुळे त्याची **value** बदलल्यावर ती सर्व **objects** मध्ये **reflect** होते.

3. Static Methods

static methods म्हणजे असे **methods** जे **class** ला संबंधित असतात, **object** ला नाही.

हे **methods** आपण **object** तयार न करता **call** करू शकतो.

Static Method चे नियम:

this किंवा **super** वापरू शकत नाहीत

non-static variables direct access करू शकत नाहीत

फक्त **static variables** किंवा इतर **static methods** **access** करू शकतात

उदाहरण:

```
class Counter {
    private static int count = 0;

    public static void increment() {
        count++;
    }

    public static int getCount() {
        return count;
    }
}
```

स्पष्टीकरण:

increment() ही **static method** असल्यामुळे आपण ती **class** नावाने **call** करू शकतो.

4. Static Blocks

static block चा वापर **static variables initialize** करण्यासाठी होतो.

हा **block class load** होताच **execute** होतो आणि फक्त एकदाच चालतो.

उदाहरण:

```
class Config {  
    static String url;  
    static int timeout;  
  
    static {  
        url = "jdbc:mysql://localhost:3306/test";  
        timeout = 30;  
        System.out.println("Static Block Executed");  
    }  
}
```

स्पष्टीकरण:

Config class JVM मध्ये **load** होताच **static block** चालतो आणि **static variables initialize** होतात.

5. Static Nested Classes



static nested class म्हणजे एखादी **class** जी दुसऱ्या **class** च्या आत **static** म्हणून **define** केली जाते.

ती **outer class** च्या फक्त **static members access** करू शकते.

उदाहरण:

```
class Outer {  
    static int data = 50;  
  
    static class Inner {  
        void msg() {  
            System.out.println("Data is: " + data);  
        }  
    }  
}
```

स्पष्टीकरण:

Inner ही **static class** असल्यामुळे ती **Outer** च्या **static variable** ला **access** करू शकते.

6. Static in Interfaces

Java 8 पासून **interfaces** मध्ये देखील **static methods** असू शकतात. हे **methods** फक्त **interface** नावानेच **call** करता येतात.

उदाहरण:

```
interface Utils {  
    static int add(int a, int b) {  
        return a + b;  
    }  
}
```

वापर:

```
System.out.println(Utils.add(5, 10));
```

7. Inheritance आणि Static Methods

static methods override होत नाहीत, त्याऐवजी त्या **hide** होतात. याला **method hiding** म्हणतात.

उदाहरण:

```
class Parent {  
    static void display() {  
        System.out.println("Parent Class");  
    }  
}  
  
class Child extends Parent {  
    static void display() {  
        System.out.println("Child Class");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Parent p = new Child();  
        p.display();    // Parent Class  
    }  
}
```

```
    Child.display(); // Child Class  
  }  
}
```

स्पष्टीकरण:

JVM ने **object** तयार केला असला तरी **p.display()** हे **Parent** चं **method call** करतं कारण **method hiding** होते.

8. main() Method Static का असते?

main() method ही **static** असते कारण **JVM** ला **object** तयार न करता ती **direct call** करता यावी.

जर ती **static** नसती तर **JVM** ला आधी **object** तयार करावा लागला असता - पण **entry point** नसल्यामुळे ते शक्य नाही.

9. Static चे फायदे (Advantages)

1. **Memory efficient** – एकच **copy** असल्यामुळे **memory** कमी लागते.
2. **Utility methods** सहज **implement** करता येतात. (उदा. **Math class**)
3. **Class-level data share** करणे सोपे जाते.

10. Static चे तोटे (Limitations)

1. **static methods** ला **instance variables direct access** करता येत नाही.
2. **Testing** आणि **flexibility** कमी होते.
3. खूप **static** वापरल्यास **code rigid** होतो (**tight coupling** वाढते).

11. मुलाखतीसाठी संभाव्य प्रश्न (Interview Questions)

Q1. Static variables memory मध्ये कुठे store होतात?
Method Area (किंवा **Java 8** नंतर: **MetaSpace**)

Q2. Static methods override होतात का?
नाही. ते **hidden** असतात, **override** होत नाहीत.

Q3. Static variables ची default value काय असते?

int → 0

boolean → false

object reference → null

Q4. जर main method वरून static काढलं तर काय होईल?
JVM program run करू शकणार नाही कारण main method non-static असल्यास object लागेल, पण JVM कडे entry point नसतो.

तयार आहात का **Static** विषयावर **interview** देण्यासाठी?

